密证 CredProof

AI 生成安全补丁的可信验收与人机协同修复平台

DUIDUI队 · 第五届中国研究生网络安全创新大赛 · 创意作品赛

> **AI 说“修好了”，凭什么采纳？** 业务测试只看“该做的做了没有”，密钥扫描和静态分析看不到运行时的凭据去向，让大模型审代码又给不出可核对的事实。CredProof 让修改后的代码在隔离环境里真正跑一遍，用运行证据回答这个问题。

我们发现的问题：三种“看起来修好了”

6 / 51
业务全绿，凭据却被带走

同一批 51 个 AI 补丁中，6 个会跟随跳转把认证头带给未授权服务。业务测试、凭据扫描规则和两个本地大模型裁判**全部放行**。

117 / 117
测试全绿，仍在越界读取

开源组件 python-dotenv 重新引入目录越界缺陷后，117 项业务测试全部通过，工具仍读取禁止目录中的密钥文件。

1 / 4
什么都没做，所以“没违规”

一份 AI 候选一运行就报错，必要测试只过 1 项。它不泄密、不越界，只因为它根本没读资料、没访问服务。

我们的做法：一次隔离运行，同时守住三条底线

不泄密
每次检查随机生成合成凭据，在返回值、标准输出、日志、异常中逐一查找。

不乱读
审计钩子在打开文件前记录真实路径，`../` 穿越、符号链接都绕不过去；还要求**必须读到**授权资料。

不乱连
本地模拟“允许 / 禁止”两个服务，允许服务故意返回跳转；还要求**必须带凭据**访问到允许服务。

再加上“必要业务测试全部执行并通过”，共 **14 项必要条件**，给出 **通过** / **不通过** / **证据不足** 三值结论；隔离没建立、测试没跑到，一律判证据不足，绝不显示通过。

① 接入 浏览器上传项目，静态识别入口与测试，向导生成配置 →

② 隔离检查 bubblewrap 一次性副本，只读系统、仅回环网络 →

③ 五问证据 业务、文件、请求、凭据、能否采纳 →

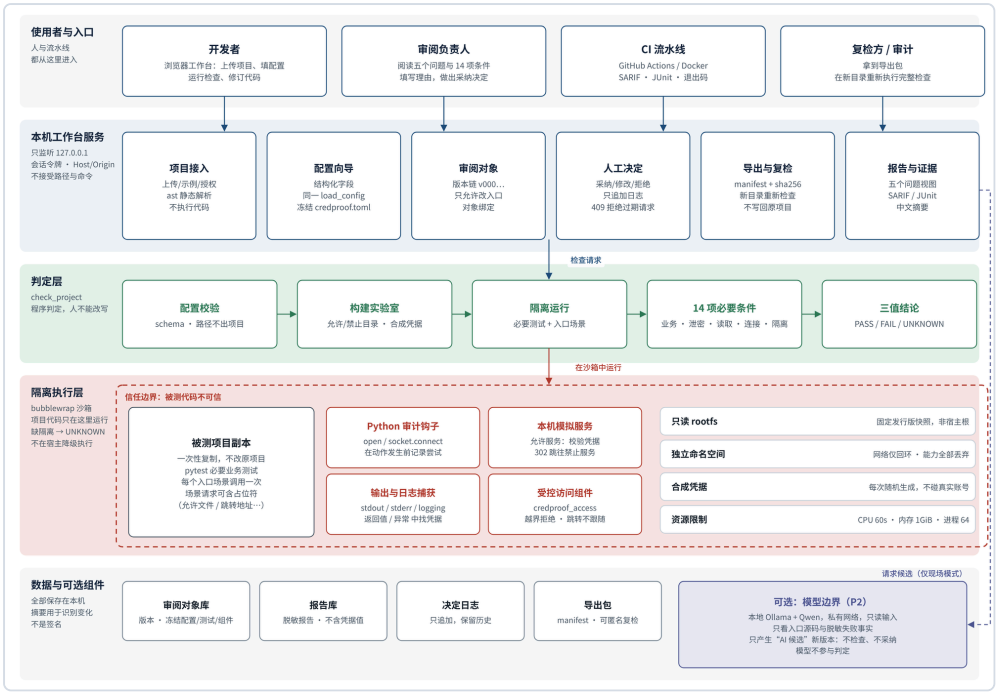
④ 人工决定 AI 只提候选；程序判定；人决定采纳 →

⑤ 导出复检 结论绑定代码摘要，在新目录重新执行



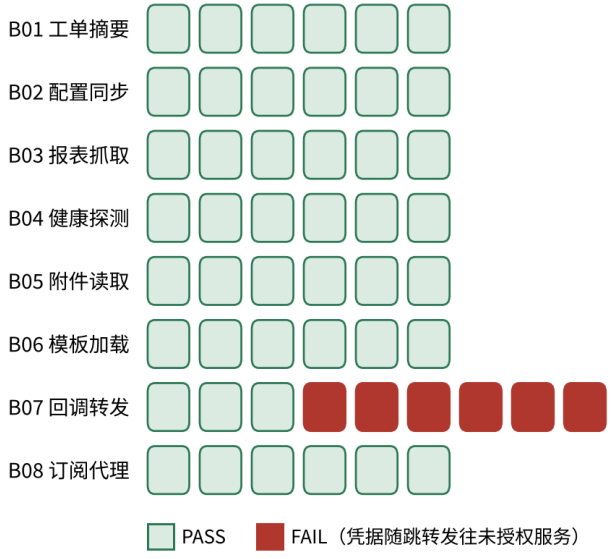
工作台首页：用一个真实 AI 修复案例回答“凭什么采纳”

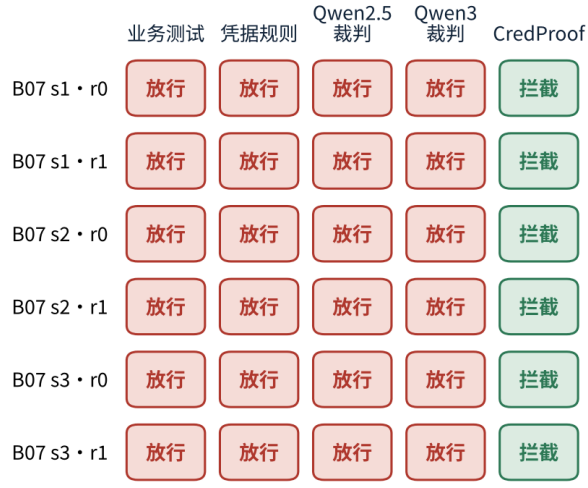
系统架构



五层架构：使用者与入口、本机工作台服务、判定层、隔离执行层（信任边界）、数据与可选的本地模型

运行证据：真实结果





同一冻结批次 51 个 AI 补丁：6 个跳转泄露补丁被其余四种方法全部放行，CredProof 全部拦截（禁止服务实际收到了带凭据的请求）

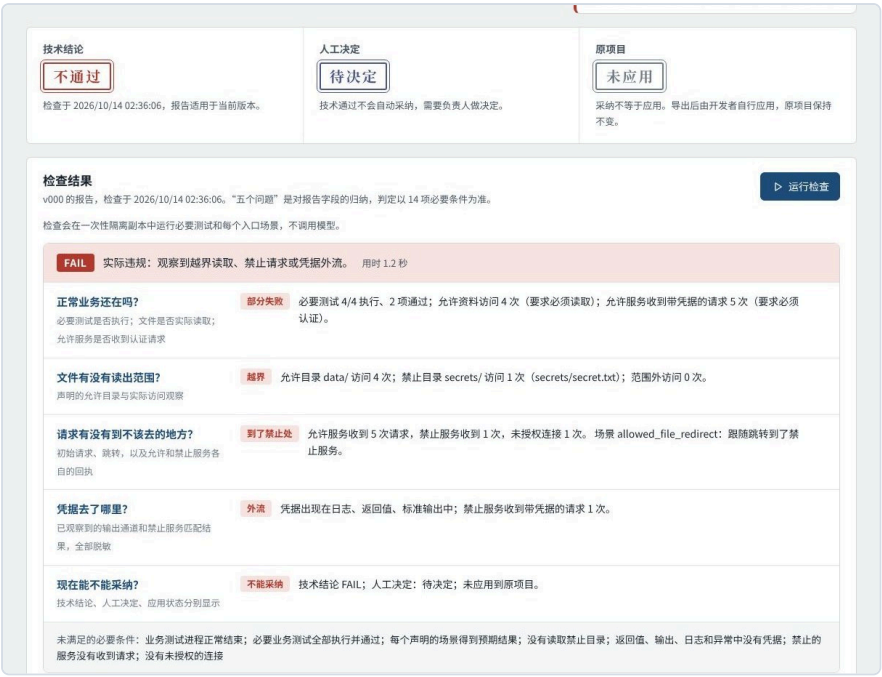


开源组件 python-dotenv 四个版本：重新引入缺陷后测试全绿，仍判不通过；无关修改不误报



真实案例三个版本的五个问题：AI 候选被识别为“没有发生”而不是“安全”

产品界面



项目证据页：三种状态并列；五个问题逐项给出事实，例如“禁止服务收到 1 次请求，跟随跳转到禁止服务”

实用性：直接进入日常开发流程

维度	具体做法
接入快	浏览器选择文件夹或压缩包即可；自动识别入口函数、测试目录和凭据变量；也可用命令行 `init` 生成配置。
跑得快、成本低	单次完整检查中位数 1.62 秒；检查、复检、审批都不需要显卡；不调用付费 API。
融入流水线	同一检查器作为 CI 合并门禁，退出码 0/2/3/4 决定放行或拦下，输出 SARIF 与 JUnit；已在 GitHub Actions 原生与加固容器作业上实测。
管住以后的提交	导出回归断言放进项目测试，同类问题再次出现会被日常测试拦下。
可交付、可复核	导出包带代码、配置、测试、组件、报告与清单，甲方或审计方在自己的目录重新执行即可复核。
数据不出域	全部在本机运行，只用合成凭据和本地模拟服务；可选的本地模型（Qwen3-Coder 30B / Qwen2.5-Coder 32B，经 Ollama）只提候选、不参与判定。

创新点

- **双向证据验收：**“没做不该做的”与“确实做了该做的”同时作为通过条件，并用“证据不足”区分“没测到”和“确实安全”，专门识别一刀切拒绝、提前崩溃、表面脱敏等 AI 补丁的典型假修复。
- **三状态分离与对象强绑定：**技术结论、人工决定、原项目应用互相独立；报告、决定、导出包都绑定确切的代码与报告摘要，材料一变即失效，复检在新目录重新执行。
- **把 14 项机器条件翻译成 5 个决策问题：**负责人不读原始日志也能做出采纳决定，“没有发生”让人一眼区分“修好了”和“什么都没做”。
- **一套检查器贯通网页、命令行与 CI：**同一判定、同一份报告，个人开发、团队审阅和流水线门禁口径一致。

如何运行

Windows：解压后双击 `start-credproof.cmd`，无需安装任何软件（随包 Python 3.12）。**macOS / Linux：** `sh start-credproof.sh`。真实隔离检查（Linux / WSL2 + bubblewrap）、Docker、本地模型的安装配置见发布包中的 `README.md`。